Desarrollo de aplicaciones móviles I

Tema Nº5:
Componentes Básicos de Android.

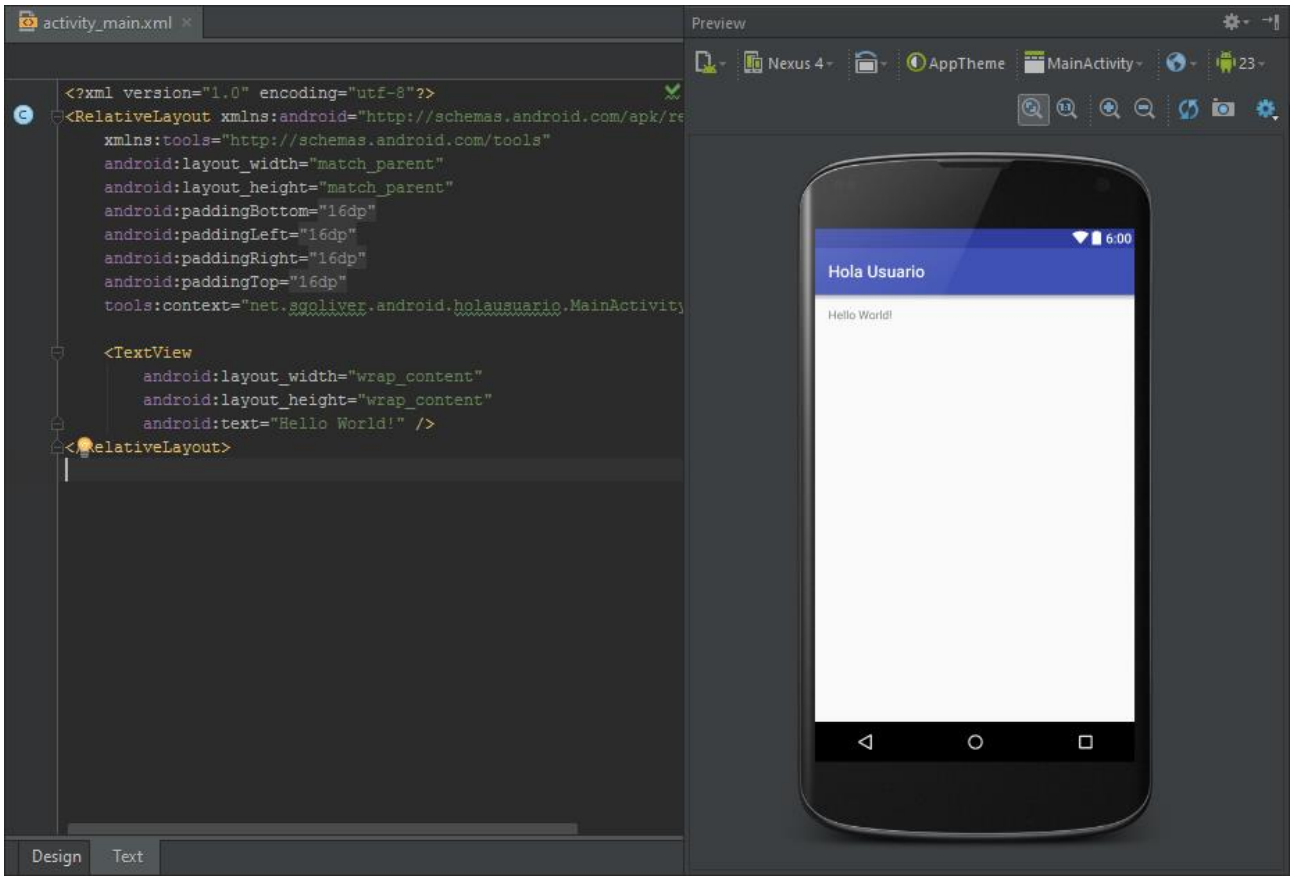
Indicador de logro Nº5:
Aplica los componentes básicos de la interfaz de usuario de una aplicación en Android a través de la implementación de una aplicación móvil que integren los elementos más comunes.

idat

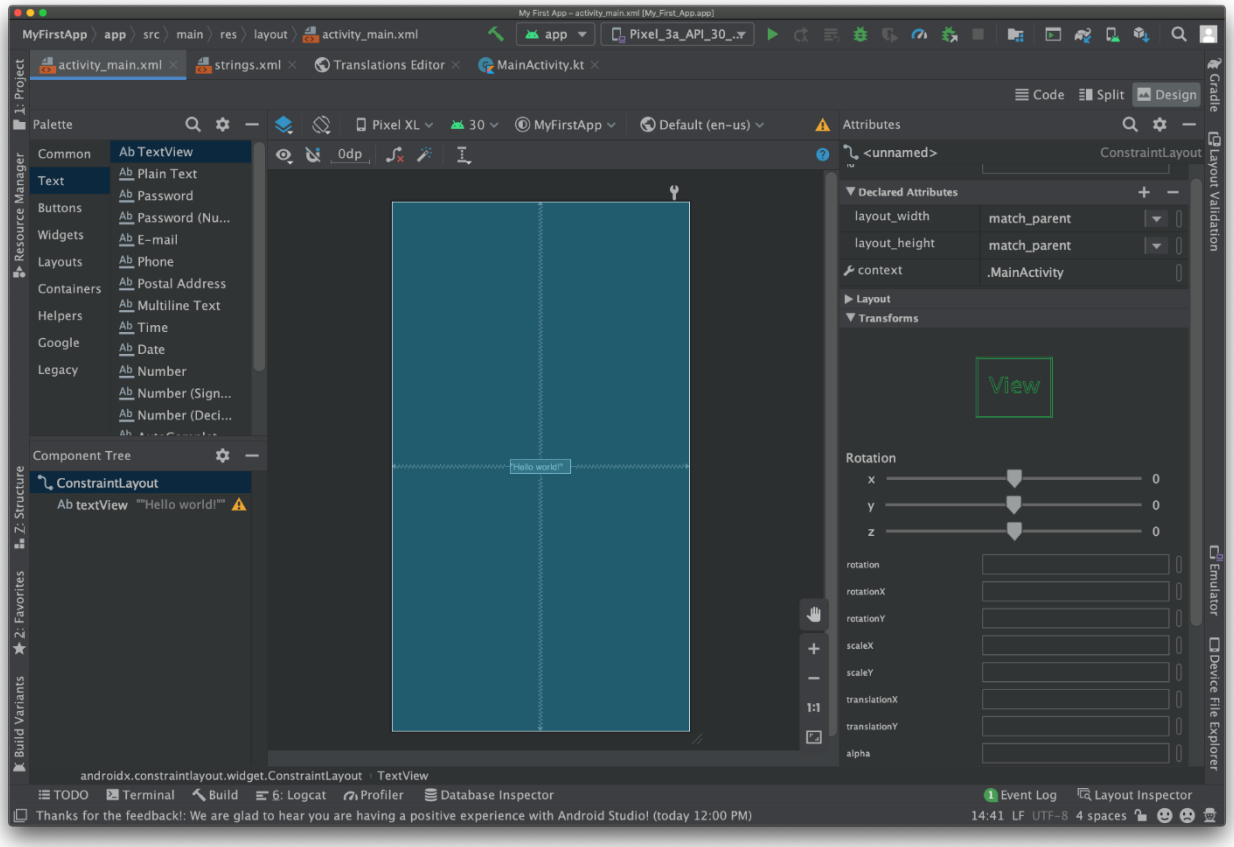
TEMA Nº5:
Creando nuestro entorno de desarrollo.

Subtema 5.1:
XML vs. Editor Gráfico
Ejemplos:

Editor XML.



Editor gráfico.



Subtema 5.2:

Diferencia entre LinearLayout y Relative Layout.

Ejemplos:

FrameLayout

Éste es el más simple de todos los layouts de Android. Un FrameLayout coloca todos sus controles hijos alineados con su esquina superior izquierda, de forma que cada control quedará oculto por el control siguiente (a menos que éste último tenga transparencia). Por ello, suele utilizarse para mostrar un único control en su interior, a modo de contenedor (placeholder) sencillo para un sólo elemento sustituible, por ejemplo una imagen.

Los componentes incluidos en un FrameLayout podrán establecer sus propiedades android:layout_width y android:layout_height, que podrán tomar los valores «match_parent» (para que el control hijo tome la dimensión de su layout contenedor) o «wrap_content» (para que el control hijo tome la dimensión de su contenido).

Veamos un ejemplo donde colocamos un FrameLayout que ocupará toda la pantalla (estableciendo como «match_parent» tanto su ancho como su alto) y en su interior un botón (Button) con el ancho ajustado a su contenedor («match_parent«) y el alto a su contenido («wrap_content«):

```
1.  <FrameLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent">
5.
6.      <Button android:id="@+id/txtBoton1"
7.          android:layout_width="match_parent"
8.          android:layout_height="wrap_content"
9.          android:text="Botón 1" />
10.
11. </FrameLayout>
```

Con el código anterior conseguimos un layout tan sencillo como el siguiente:

LinearLayout

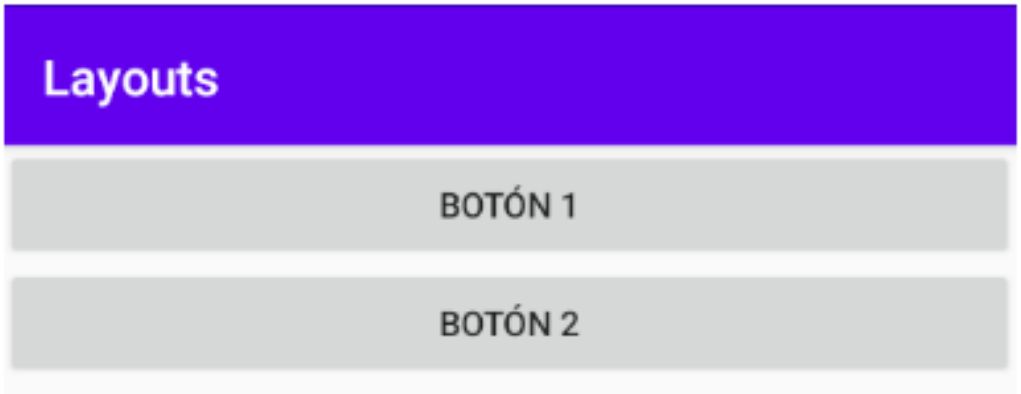
El siguiente tipo de layout en cuanto a nivel de complejidad es el LinearLayout. Este layout apila uno tras otro todos sus elementos hijos en sentido horizontal o vertical según se establezca su propiedad android:orientation.

Al igual que en un FrameLayout, los elementos contenidos en un LinearLayout pueden establecer sus propiedades android:layout_width y android:layout_height para determinar sus dimensiones dentro del layout.

Veamos un ejemplo donde colocamos dos botones dentro de un LinearLayout con orientación vertical:

```
1.  <LinearLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent"
5.      android:orientation="vertical">
6.
7.      <Button android:id="@+id/txtBoton1"
8.          android:layout_width="match_parent"
9.          android:layout_height="wrap_content"
10.         android:text="Botón 1" />
11.
12.     <Button android:id="@+id/txtBoton2"
13.         android:layout_width="match_parent"
14.         android:layout_height="wrap_content"
15.         android:text="Botón 2" />
16.
17. </LinearLayout>
```

El layout anterior quedaría de la siguiente forma:

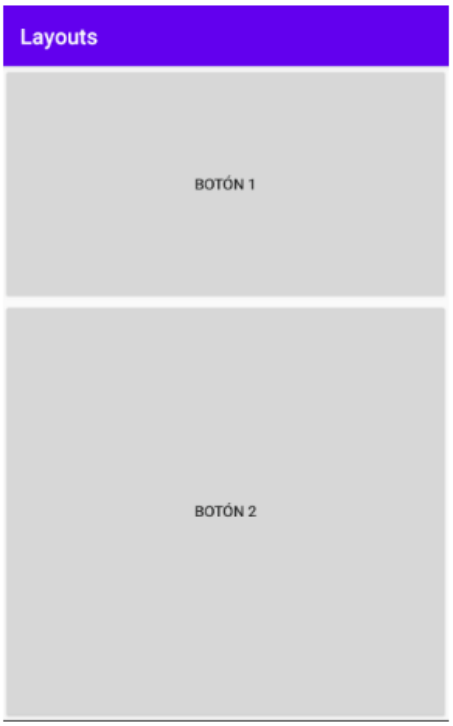


Pero en el caso de un LinearLayout, tendremos otro parámetro con el que jugar, la propiedad android:layout_weight. Esta propiedad nos va a permitir dar a los elementos contenidos en el layout unas dimensiones proporcionales entre ellas.

Esto es más difícil de explicar que de comprender con un ejemplo. Si incluimos en un LinearLayout vertical dos botones (Button) y a uno de ellos le establecemos un layout_weight="1" y al otro un layout_weight="2" conseguiremos como efecto que toda la superficie del layout quede ocupada por los dos botones y que además el segundo sea el doble (relación entre sus propiedades weight) de alto que el primero.

```
1.  <LinearLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent"
5.      android:orientation="vertical">
6.
7.      <Button android:id="@+id/txtBoton1"
8.          android:layout_width="match_parent"
9.          android:layout_height="wrap_content"
10.         android:layout_weight="1"
11.         android:text="Botón 1" />
12.
13.     <Button android:id="@+id/txtBoton2"
14.         android:layout_width="match_parent"
15.         android:layout_height="wrap_content"
16.         android:layout_weight="2"
17.         android:text="Botón 2" />
18.
19. </LinearLayout>
```

Con el código anterior conseguiríamos un layout como el siguiente:



Así pues, a pesar de la simplicidad aparente de este layout resulta ser lo suficiente versátil como para sernos de utilidad en muchas ocasiones.

TableLayout

Un TableLayout permite distribuir sus elementos hijos de forma tabular, definiendo las filas y columnas necesarias, y la posición de cada componente dentro de la tabla.

La estructura de la tabla se define de forma similar a como se hace en HTML, es decir, indicando las filas que compondrán la tabla (objetos TableRow), y dentro de cada fila las columnas necesarias, con la salvedad de que no existe ningún objeto especial para definir una columna (algo así como un TableColumn) sino que directamente insertaremos los controles necesarios dentro del TableRow y cada componente insertado (que puede ser un control sencillo o incluso otro ViewGroup) corresponderá a una columna de la tabla. De esta forma, el número final de filas de la tabla se corresponderá con el número de elementos TableRow insertados, y el número total de columnas quedará determinado por el número de componentes de la fila que más componentes contenga.

Por norma general, el ancho de cada columna se corresponderá con el ancho del mayor componente de dicha columna, pero existen una serie de propiedades que nos ayudarán a modificar este comportamiento:

- android:stretchColumns. Indicará las columnas que pueden expandir para absorber el espacio libre dejado por las demás columnas a la derecha de la pantalla.
- android:shrinkColumns. Indicará las columnas que se pueden contraer para dejar espacio al resto de columnas que se puedan salir por la derecha de la pantalla.
- android:collapseColumns. Indicará las columnas de la tabla que se quieren ocultar completamente.

Todas estas propiedades del TableLayout pueden recibir una lista de índices de columnas separados por comas (ejemplo: android:stretchColumns="1,2,3") o un asterisco para indicar que debe aplicar a todas las columnas (ejemplo: android:stretchColumns="*").

Otra característica importante es la posibilidad de que una celda determinada pueda ocupar el espacio de varias columnas de la tabla (análogo al atributo colspan de HTML). Esto se indicará mediante la propiedad android:layout_span del componente concreto que deberá tomar dicho espacio.

Veamos un ejemplo con varios de estos elementos:

```
1.  <TableLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent"
5.      android:stretchColumns="0,1,2">
6.
7.      <TableRow>
8.          <TextView android:text="Celda 1.1" />
9.          <TextView android:text="Celda 1.2" />
10.         <TextView android:text="Celda 1.3" />
11.     </TableRow>
12.
13.     <TableRow>
14.         <TextView android:text="Celda 2.1" />
15.         <TextView android:text="Celda 2.2" />
16.         <TextView android:text="Celda 2.3" />
17.     </TableRow>
18.
19.     <TableRow>
20.         <TextView android:text="Celda doble de ancho 3.1"
21.             android:layout_span="2" />
22.         <TextView android:text="Celda 3.2" />
23.     </TableRow>
24. </TableLayout>
```

El layout resultante del código anterior sería el siguiente:

GridLayout

Sus características son similares al TableLayout, ya que se utiliza igualmente para distribuir los diferentes elementos de la interfaz de forma tabular, distribuidos en filas y columnas. La diferencia entre ellos estriba en la forma que tiene el GridLayout de colocar y distribuir sus elementos hijos en el espacio disponible. En este caso, a diferencia del TableLayout indicaremos el número de filas y columnas como propiedades del layout, mediante android:rowCount y android:columnCount. Con estos datos ya no es necesario ningún tipo de elemento para indicar las filas, como hacíamos con el elemento TableRow del TableLayout, sino que los diferentes elementos hijos se irán colocando ordenadamente por filas o columnas (dependiendo de la propiedad android:orientation) hasta completar el número de filas o columnas indicadas en los atributos anteriores. Adicionalmente, igual que en el caso anterior, también tendremos disponibles las propiedades android:layout_rowSpan y android:layout_columnSpan para conseguir que una celda ocupe el lugar de varias filas o columnas. Por su parte, para definir cómo ocupar el espacio horizontal disponible ocupado por las columnas debemos jugar con la propiedad layout_columnWeight de cada elemento hijo.

Existe también una forma de indicar de forma explícita la fila y columna que debe ocupar un determinado elemento hijo contenido en el GridLayout, y se consigue utilizando los atributos android:layout_row y android:layout_column. De cualquier forma, salvo para configuraciones complejas del grid no suele ser necesario utilizar estas propiedades.

Con todo esto en cuenta, para conseguir una distribución equivalente a la del ejemplo anterior del TableLayout, necesitaríamos escribir un código como el siguiente:

```
1.  <GridLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent"
5.      android:rowCount="2"
6.      android:columnCount="3"
7.      android:orientation="horizontal">
8.
9.      <TextView android:layout_columnWeight="1" android:text="Celda 1.1" />
10.     <TextView android:layout_columnWeight="1" android:text="Celda 1.2" />
11.     <TextView android:layout_columnWeight="1" android:text="Celda 1.3" />
12.
13.     <TextView android:layout_columnWeight="1" android:text="Celda 2.1" />
14.     <TextView android:layout_columnWeight="1" android:text="Celda 2.2" />
15.     <TextView android:layout_columnWeight="1" android:text="Celda 2.3" />
16.
17.     <TextView android:layout_columnWeight="1" android:text="Celda doble de anch
18.         android:layout_columnSpan="2" />
19.     <TextView android:layout_columnWeight="1" android:text="Celda 3.2" />
20.
21. </GridLayout>
```

RelativeLayout

El último tipo de layout que vamos a ver es el RelativeLayout. Este layout permite especificar la posición de cada elemento de forma relativa a su elemento padre o a cualquier otro elemento incluido en el propio layout. De esta forma, al incluir un nuevo elemento X podremos indicar por ejemplo que debe colocarse debajo del elemento Y y alineado a la derecha del layout padre. Veamos esto en el ejemplo siguiente:

```
1.  <RelativeLayout
2.      xmlns:android="http://schemas.android.com/apk/res/android"
3.      android:layout_width="match_parent"
4.      android:layout_height="match_parent" >
5.
6.      <EditText android:id="@+id/txtNombre"
7.          android:layout_width="match_parent"
8.          android:layout_height="wrap_content"
9.          android:inputType="text" />
10.
11.     <Button android:id="@+id/btnAceptar"
12.         android:layout_width="wrap_content"
13.         android:layout_height="wrap_content"
14.         android:layout_below="@id/txtNombre"
15.         android:layout_alignParentRight="true"
16.         android:text="Aceptar" />
17.
18. </RelativeLayout>
```

En el ejemplo, el botón (Button) se colocará debajo del cuadro de texto (EditText) dado que se ha establecido la propiedad android:layout_below="@id/txtNombre" y alineado a la derecha del layout padre al establecer android:layout_alignParentRight="true", Quedaría algo así:

Al igual que estas tres propiedades, en un RelativeLayout tendremos un sinfín de propiedades para colocar cada control justo donde queramos. Veamos las principales [creo que sus propios nombres explican perfectamente la función de cada una]:

Posición relativa a otro control:

- android:layout_above
- android:layout_below
- android:layout_toLeftOf
- android:layout_toRightOf
- android:layout_alignLeft
- android:layout_alignRight
- android:layout_alignTop
- android:layout_alignBottom
- android:layout_alignBaseline

Posición relativa al layout padre:

- android:layout_alignParentLeft
- android:layout_alignParentRight
- android:layout_alignParentTop

- android:layout_alignParentBottom
- android:layout_centerHorizontal
- android:layout_centerVertical
- android:layout_centerInParent

Por último indicar que cualquiera de los tipos de layout anteriores poseen otras propiedades comunes como por ejemplo los márgenes exteriores (margin) e interiores (padding) que pueden establecerse mediante los siguientes atributos:

Opciones de margen exterior:

- android:layout_margin
- android:layout_marginBottom
- android:layout_marginTop
- android:layout_marginLeft
- android:layout_marginRight

Opciones de margen interior:

- android:padding
- android:paddingBottom
- android:paddingTop
- android:paddingLeft
- android:paddingRight

ConstraintLayout

Actualmente es el layout recomendado por Google para definir cualquier interfaz de cierta complejidad. Aunque con los tipos de layout descritos hasta este momento en esta entrada podríamos definir casi cualquier interfaz, habitualmente anidando unos dentro de otros, el nuevo ConstraintLayout tiene la ventaja de ser capaz de conseguir los mismos resultados sin necesidad de anidar diferentes layouts, es decir, con un árbol o jerarquía de componentes mucho más «plana», y por tanto más eficiente.

La ventaja que ofrece este Layout a la hora de diseñarlo es que puedes poner una relación de cada elemento con otros elementos o con el mismo Layout haciéndolo totalmente responsive, teniendo la seguridad de que el control o elemento se mantendrá en su posición relativa dentro de la pantalla sea cual sea su tamaño.

```xml
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.mejorprogramacion.layouts.Constraint">
    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="mejorprogramacion.com" />
    <TextView
        android:id="@+id/textView"
        android:layout_width="299dp"
        android:layout_height="33dp"
        android:layout_marginEnd="40dp"
        android:layout_marginStart="8dp"
        android:text="TextView"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        tools:layout_editor_absoluteY="41dp" />
    <EditText
        android:id="@+id/editText"
        android:layout_width="148dp"
        android:layout_height="52dp"
        android:layout_marginBottom="8dp"
        android:layout_marginEnd="8dp"
        android:layout_marginStart="8dp"
        android:layout_marginTop="20dp"
        android:ems="5"
        android:inputType="textPersonName"
        android:text="Name"
        app:layout_constraintBottom_toTopOf="@+id/button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.881"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/textView"
        app:layout_constraintVertical_bias="0.464" />
```

```xml
    <Button
        android:id="@+id/button"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:layout_marginTop="168dp"
        android:text="Button"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/textView"
        tools:layout_editor_absoluteX="0dp" />
</android.support.constraint.ConstraintLayout>
```

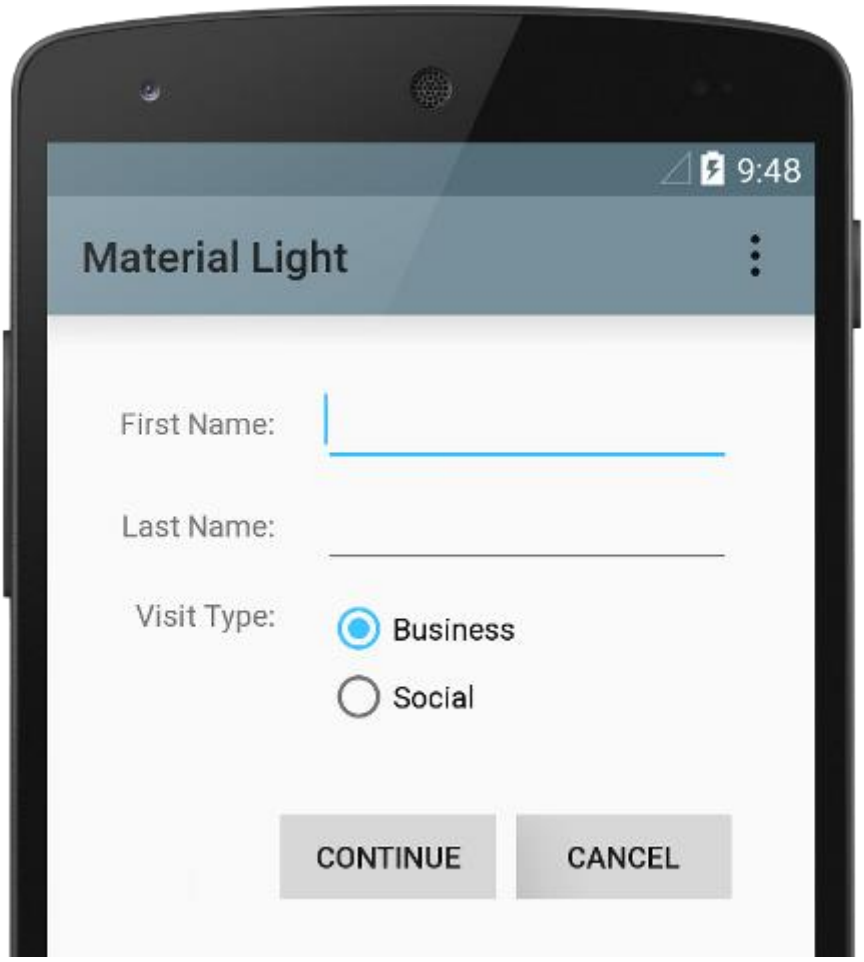
Subtema 5.3:

Widgets Básicos en Android.

Ejemplos:

Entre los principales widgets tenemos:

- TextView: Etiquetas de texto.
- EditText: Caja de texto.
- RadioButton: Selección de opción.
- Button: Botón para una acción.



Actividad:

a) CUESTIONARIO TÉCNICO

- ¿Cuál es la finalidad de utilizar layouts en nuestros diseños de interfaces de usuario?
- Detalle la diferencia entre in TableLayout y un GridLayout
- Indique la utilidad de las propiedades android:layout_row y android:layout_column al trabajar con un Grid_Layout

- Indique cuándo se utiliza la propiedad match_parent al trabajar con Layouts
- ¿Por qué Google recomienda el uso de Constrainr Layout al diseñar las interfaces de usuario?

b) CONCLUSIONES DE LA EXPERIENCIA

_____
_____
_____
_____
_____
_____
_____